DATABASE MIGRATION

QuranFlow

Schema Migration

Old schema (82 tables) → New unified schema (22 core tables)

82	22	60
old tables	new tables	eliminated

Clusters affected
Content · Delivery · Responses

New core tables
qf_config_keys, qf_config, qf_delivery_types
qf_delivery, qf_payload, qf_resource
qf_delivery_payload, qf_content_delivery
qf_student_responses, qf_delivery_responses

Key principle
qf_config is the universal EAV store.
All flexible metadata lives there.

All Features — Old Schema

Every table in the original 82-table schema, grouped by cluster

Content	Delivery	Responses	Dropped (logs)	Unchanged
lessons	assessment	answer	debug_logs	semester
lesson_link_video	assessment_link_question	student_submitted_assessment	error_logs	users
lesson_semester_plans	assessment_semester_plans	student_submitted_assessment_link_answer	log	user_profile
resource	question	assessment_score	request_log	user_link_level_tag
resource_semester_plan	quiz	feedback	isoft_log	group_name
ta_recordings	quiz_mcq_question	feedback_parameters	internal_launch_automation_logs	group_chat
tutorials	quiz_semester	quiz_score	in_app_purhcase_logs	group_members
tutorial_semester_plan	mcq_question	quiz_user_link_missing_level_tag	test	payments
welcome_videos	mcq_question_option	user_link_lesson		payment_plans
electives	ta_live_sessions	notifications_settings_new		coupons
support_links	ta_meeting_room_links	users_notifications_settings_new		subscriptions
email_templates	ta_schedule_timing	user_clear_notifications		student_ta_appointment
mail_merger				ta_holidays
page_content				level_tag
notes				tags
15 tables	12 tables	12 tables	8 tables	19 tables

Content Cluster

15 tables → qf_delivery + qf_resource + qf_payload + qf_config

CONTENT | MIGRATED

lessons

↳ *qf_delivery (type = 'lesson')*

Each lesson becomes one delivery row. Scheduling, audience, and elective metadata move to qf_config.

Column	Type	Migration destination / note
PK ls_id	PK	→ qf_delivery.delivery_id
ls_description	text	→ qf_delivery.title
ls_level_tags	varchar	→ config: level_tags
ls_posted_date	date	→ config: post_date
ls_posted_days	int	→ config: post_week
is_understand	tinyint	→ config: is_understand_required
ls_created_at	datetime	→ qf_delivery.created_at
ls_updated_at	datetime	→ qf_delivery.updated_at
elective_id	int	→ config: elective_id + elective_name
is_core_lesson	tinyint	→ config: is_core

CONTENT MIGRATED

lesson_link_video

↳ *qf_resource (resource_type=video) + qf_content_delivery bridge*

Each video becomes a qf_resource row. The lesson↔video relationship becomes a qf_content_delivery row.

Column	Type	Migration destination / note
PK llv_id	PK	→ qf_resource.resource_id
FK ls_id	FK	→ qf_content_delivery.delivery_id
llv_title	varchar	→ config: display_name
llv_link	varchar	→ config: link_url
llv_link_thumb	varchar	→ config: thumbnail_url
llv_description	text	→ config: resource_description
llv_levels	varchar	→ config: visible_to_designation
llv_posted_days	int	→ config: post_week
llv_posted_date	date	→ config: available_from
llv_custom_thumb	varchar	→ config: custom_thumbnail_url

CONTENT MIGRATED

lesson_semester_plans

↳ *qf_delivery.semester_id (direct column) — bridge table eliminated*

Semester scoping was done via a separate bridge. Now semester_id is a direct column on qf_delivery.

Column	Type	Migration destination / note
PK lesson_plan_id	PK	DROPPED — bridge eliminated
semester_id	int	→ qf_delivery.semester_id
FK lesson_id	FK	→ resolved to qf_delivery.delivery_id

CONTENT | MIGRATED

resource

↳ *qf_resource (resource_type=attachment) + qf_content_delivery*

Standalone file/link resources become qf_resource rows. All metadata (title, description, type, order) moves to qf_config.

Column	Type	Migration destination / note
PK id	PK	→ qf_resource.resource_id
res_title	varchar	→ config: display_name
res_description	text	→ config: resource_description
res_link	varchar	→ config: link_url
res_file	varchar	→ qf_resource.attachment_id (file store ref)
res_created_at	datetime	→ qf_resource.created_at
res_updated_at	datetime	→ qf_resource.updated_at
res_type	varchar	→ config: resource_type
res_order	int	→ config: sequence
elective_id	int	→ config: elective_id
is_core_resource	tinyint	→ config: is_core

CONTENT | MIGRATED

resource_semester_plan

↳ *qf_content_delivery (delivery_id + resource_id) + qf_config for scoping*

Column	Type	Migration destination / note
PK resource_plan_id	PK	DROPPED — bridge eliminated
semester_id	int	→ config: visible_to_semester_id on qf_resource
FK resource_id	FK	→ qf_resource.resource_id
res_level_tags	varchar	→ config: visible_to_designation

CONTENT | MIGRATED

ta_recordings

↳ *qf_delivery (type = 'ta_recording') — new delivery type added*

TA recordings were a standalone island. They become a delivery type, enabling consistent scheduling and audience scoping.

Column	Type	Migration destination / note
PK id	PK	→ qf_delivery.delivery_id
r_title	varchar	→ qf_delivery.title
r_description	text	→ qf_delivery.description
r_level_tags	varchar	→ config: level_tags
r_link	varchar	→ config: session_link
r_date	date	→ config: post_date
r_teaching_assistant	int	→ config: teaching_assistant_id
r_recording_type	varchar	→ config: recording_type
gender_scope	varchar	→ config: session_gender_scope
elective_id	int	→ config: elective_id
is_core_recording	tinyint	→ config: is_core

CONTENT | MIGRATED

tutorials

↳ *qf_delivery (type = 'tutorial') + qf_resource for files/links*

Column	Type	Migration destination / note
PK id	PK	→ qf_delivery.delivery_id
tut_title	varchar	→ qf_delivery.title
tut_description	text	→ qf_delivery.description
tut_link	varchar	→ qf_resource (resource_type=link) + qf_content_delivery
tut_file	varchar	→ qf_resource.attachment_id
tut_video_link	varchar	→ qf_resource (resource_type=video)
tut_type	varchar	→ config: tutorial_type

CONTENT | MIGRATED

tutorial_semester_plan

↳ *qf_delivery.semester_id + config: designation_id — bridge eliminated*

Column	Type	Migration destination / note
PK tutorial_plan_id	PK	DROPPED — bridge eliminated
semester_id	int	→ qf_delivery.semester_id
FK tutorial_id	FK	→ resolved to qf_delivery.delivery_id
level_tag_id	int	→ config: designation_id

CONTENT | MIGRATED

welcome_videos

↳ *qf_delivery (type = 'welcome_video') + qf_resource for video/thumbnail*

Column	Type	Migration destination / note
PK id	PK	→ qf_delivery.delivery_id
title	varchar	→ qf_delivery.title
description	text	→ qf_delivery.description
video_url	varchar	→ qf_resource (resource_type=video) + qf_content_delivery
thumbnail_url	varchar	→ qf_resource (resource_type=thumbnail)
level_tag_id	int	→ config: designation_id
elective_id	int	→ config: elective_id
is_year2_core	tinyint	→ config: is_core
is_active	tinyint	→ config: is_active
display_order	int	→ config: sequence

CONTENT | DISSOLVED

electives

↳ *DISSOLVED → config keys on qf_delivery — no separate table*

Electives no longer exist as a separate entity. The name and is_practice_only flag are denormalised into config keys on each delivery row.

Column	Type	Migration destination / note
PK `id`	PK	DISSOLVED — no target row
`name`	`varchar`	→ config: elective_name (stored inline per delivery)
`is_practice_only`	`tinyint`	→ config: is_practice_only

CONTENT MIGRATED

support_links

↳ *qf_resource (resource_type = 'support_link')*

Support links (Telegram etc.) become virtual resources — attachment_id is NULL, all attributes live in qf_config.

Column	Type	Migration destination / note
PK id	PK	→ qf_resource.resource_id
name	varchar	→ config: display_name
telegram_link	varchar	→ config: link_url + platform='telegram'
year	int	→ config: target_year
gender	varchar	→ config: gender_scope
is_active	tinyint	→ config: is_active

CONTENT MIGRATED

email_templates

↳ *qf_payload (type = 'email_template') — delivery type 'email_template'*

The HTML body becomes qf_payload.body. Subject and name become config keys. Mail variables link via qf_delivery_payload.

Column	Type	Migration destination / note
PK id	PK	→ qf_payload.payload_id
subject	varchar	→ config: email_subject
name	varchar	→ config: template_name
html	mediumtext	→ qf_payload.body

CONTENT MIGRATED

mail_merger

↳ *qf_payload (type = 'mail_variable') linked via qf_delivery_payload*

Column	Type	Migration destination / note
PK id	PK	→ qf_payload.payload_id
name	varchar	→ config: merger_variable_name
type	varchar	→ config: merger_variable_type
testing_replacement_text	text	→ config: merger_testing_value

CONTENT MIGRATED

page_content

↳ *qf_payload (type = 'page_content') — delivery type 'page_content'*

Column	Type	Migration destination / note
PK id	PK	→ qf_payload.payload_id
page_key	varchar	→ config: page_key
title	varchar	→ config: page_title
content	text	→ qf_payload.body

CONTENT MIGRATED

notes

↳ *qf_payload (type = 'note') — delivery type 'note'*

Column	Type	Migration destination / note
PK id	PK	→ qf_payload.payload_id
body	text	→ qf_payload.body

Delivery Cluster

12 tables → qf_delivery + qf_payload + qf_delivery_payload + qf_config

DELIVERY | MIGRATED

assessment

↳ *qf_delivery (type = 'assessment')*

Title/description become delivery columns. Scheduling and audience metadata move to qf_config. The question link bridge replaces assessment_link_question.

Column	Type	Migration destination / note
PK assessment_id	PK	→ qf_delivery.delivery_id
title	varchar	→ qf_delivery.title
discription	text	→ qf_delivery.description
content	text	→ qf_payload (type=body)
owner_id	int	→ qf_delivery.owner_id
assessment_week	int	→ config: assessment_week
assessment_week_day	int	→ config: assessment_week_day
level_tag_id	int	→ config: designation_id
post	tinyint	→ config: post_date
status	varchar	→ config: delivery_status

DELIVERY | MIGRATED

assessment_link_question

↳ *qf_delivery_payload (delivery_id, payload_id, sequence)*

The link bridge is replaced by the generic qf_delivery_payload bridge, which works for all delivery types.

	Column	Type	Migration destination / note
PK	task_link_question_id	PK	DROPPED — generic bridge used
FK	assessment_id	FK	→ qf_delivery_payload.delivery_id
FK	question_id	FK	→ qf_delivery_payload.payload_id

DELIVERY | MIGRATED

assessment_semester_plans

↳ *qf_delivery.semester_id — bridge eliminated*

Column	Type	Migration destination / note
PK assessment_plan_id	PK	DROPPED — bridge eliminated
FK assessment_id	FK	→ resolved to qf_delivery.delivery_id
semester_id	int	→ qf_delivery.semester_id

DELIVERY | MIGRATED

question

↳ *qf_payload (type = 'question', is_mcq = 0)*

Open-answer questions become payload rows. All structural metadata (marks, language, attachment) moves to qf_config or qf_content_delivery.

Column	Type	Migration destination / note
PK question_id	PK	→ qf_payload.payload_id
discription	text	→ qf_payload.body
total_marks	decimal	→ config: total_marks
attachment	varchar	→ qf_resource + qf_content_delivery
attachment_type	varchar	→ config: resource_type on the qf_resource row
answer_type	varchar	→ config: type
statement	text	→ qf_payload.body (merged with discription)
arabic	tinyint	→ config: arabic
instructions	text	→ config: instructions
statement_formatted	text	→ config: statement_formatted

DELIVERY | MIGRATED

quiz

↳ *qf_delivery (type = 'quiz')*

Quiz scheduling (after N items, sequence) moves to qf_config. MCQ question links become qf_delivery_payload rows.

Column	Type	Migration destination / note
PK id	PK	→ qf_delivery.delivery_id
title	varchar	→ qf_delivery.title
description	text	→ qf_delivery.description
sequence	int	→ config: sequence
afterNumberItems	int	→ config: after_n_items
level_tag_id	int	→ config: designation_id
owner_id	int	→ qf_delivery.owner_id
status	varchar	→ config: delivery_status

DELIVERY | MIGRATED

quiz_mcq_question

↳ *qf_delivery_payload (delivery_id, payload_id, sequence)*

	Column	Type	Migration destination / note
PK	id	PK	DROPPED — generic bridge used
FK	quiz_id	FK	→ qf_delivery_payload.delivery_id
FK	mcq_question_id	FK	→ qf_delivery_payload.payload_id

DELIVERY | MIGRATED

quiz_semester

↳ *qf_delivery.semester_id — bridge eliminated*

Column	Type	Migration destination / note
FK quiz_id	FK	→ resolved to qf_delivery.delivery_id
semester_id	int	→ qf_delivery.semester_id

DELIVERY | MIGRATED

mcq_question

↳ *qf_payload (type = 'question', is_mcq = 1)*

MCQ questions and open questions are now the same entity — a payload. is_mcq distinguishes them via qf_config.

Column	Type	Migration destination / note
PK id	PK	→ qf_payload.payload_id
title	varchar	→ qf_payload.body
correct_option	varchar	→ config: correct_option_id (payload_id of correct option)
type	varchar	→ config: type
question_sequence	int	→ config: question_sequence

DELIVERY | MIGRATED

mcq_question_option

↳ *qf_payload (type = 'option') — linked via qf_delivery_payload*

Each option is a payload row. The display order is qf_delivery_payload.sequence. The correct option is identified by config: correct_option_id on the question payload.

Column	Type	Migration destination / note
PK id	PK	→ qf_payload.payload_id
FK mcq_question_id	FK	→ parent question identified via qf_delivery_payload grouping
option	varchar	→ qf_payload.body
option_number	int	→ config: option_number

DELIVERY | MIGRATED

ta_live_sessions

↳ *qf_delivery (type = 'ta_live_session')*

Live sessions were an isolated island. As a delivery type they get consistent scheduling, audience scoping, and elective assignment for free.

Column	Type	Migration destination / note
PK lvs_id	PK	→ qf_delivery.delivery_id
lvs_title	varchar	→ qf_delivery.title
lvs_link	varchar	→ config: session_link
lvs_category	varchar	→ config: session_category
lvs_level_tags	varchar	→ config: level_tags
lvs_posted_date	date	→ config: post_date
lvs_posted_time	time	→ config: session_posted_time
lvs_posted_day	varchar	→ config: post_week_day
lvs_teaching_assistant	int	→ config: teaching_assistant_id
time_zone	varchar	→ config: session_time_zone
gender_scope	varchar	→ config: session_gender_scope
elective_id	int	→ config: elective_id

+ 1 more columns — see migration SQL

DELIVERY | MIGRATED

ta_meeting_room_links

↳ *qf_resource (resource_type = 'ta_profile') — one resource row per TA*

The meeting room link and schedule timing collapse into a single qf_resource row per TA, with slot config rows for each availability window.

Column	Type	Migration destination / note
PK id	PK	→ qf_resource.resource_id
ta_id	int	→ config: ta_urdd_id
link	varchar	→ config: link_url
time_zone	varchar	→ config: time_zone

DELIVERY | MIGRATED

ta_schedule_timing

↳ *qf_config slot rows on qf_resource (resource_type='ta_profile')*

Each availability slot becomes a group of qf_config rows on the TA's resource. Multiple slots = multiple slot_group_id values.

Column	Type	Migration destination / note
PK id	PK	DROPPED — config rows used per slot group
FK ta_mtr_link	FK	→ qf_resource.resource_id of the ta_profile
day	tinyint	→ config: slot_day (0=Sun…6=Sat)
start_time	time	→ config: slot_start_time
end_time	time	→ config: slot_end_time
time_zone	varchar	→ config: time_zone
valid_from	date	→ config: slot_valid_from
valid_till	date	→ config: slot_valid_till

Responses Cluster

12 tables → qf_student_responses + qf_delivery_responses + qf_config

RESPONSES | MIGRATED

answer

↳ *qf_student_responses (student_id, payload_id, student_response, response_score)*

Answers, quiz responses, and lesson views all become qf_student_responses rows. The old triple-join chain is reduced to one table.

Column	Type	Migration destination / note
PK answer_id	PK	→ qf_student_responses.student_response_id
user_id	int	→ student_id (urdd_id)
FK student_submitted_assessment_id	FK	→ resolved via qf_delivery_responses
FK question_id	FK	→ payload_id
discription	text	→ student_response
marks_obtained	decimal	→ response_score
attachment	varchar	→ stored in student_response as attachment ref
attachment_type	varchar	→ implicit from qf_resource.resource_type
statement	text	→ student_response (merged)
old_url	varchar	DROPPED — URL migration artefact
new_url	varchar	DROPPED — URL migration artefact

RESPONSES MIGRATED

student_submitted_assessment

↳ *qf_delivery_responses (delivery_id + student_response_id bridge)*

The submission record existed to tie a student to an assessment. That relationship is now qf_delivery_responses. All snapshot columns are dropped — payload is the source of truth.

Column	Type	Migration destination / note
PK student_submitted_assessment_id	PK	→ qf_delivery_responses.id
FK assessment_id	FK	→ qf_delivery_responses.delivery_id
assessment_status	varchar	→ future config key on qf_delivery_responses
title	varchar	*SNAPSHOT — DROPPED (delivery is source of truth)*
discription	text	*SNAPSHOT — DROPPED*
content	text	*SNAPSHOT — DROPPED*
owner_id	int	*SNAPSHOT — DROPPED*
assessment_week	int	*SNAPSHOT — DROPPED*
reviewed_by	int	→ qf_student_responses.grader_id
reviewed_date	date	→ qf_student_responses.graded_at
user_link_level_tag_id	int	→ qf_student_responses.student_id (resolved)

+ 1 more columns — see migration SQL

RESPONSES | DROPPED

student_submitted_assessment_link_answer

↳ *DROPPED — entire table eliminated. Snapshot bridge no longer needed.*

This table stored a full snapshot of question data at submission time. Since qf_payload is immutable after publish, no snapshot is needed. All data is read directly from the payload.

Column	Type	Migration destination / note
PK id	PK	DROPPED
FK student_submitted_assessment_id	FK	DROPPED — snapshot redundancy
question_id	int	DROPPED — read from qf_payload via qf_delivery_payload
question_discription	text	DROPPED — read from qf_payload.body
question_total_marks	decimal	DROPPED — read from config: total_marks
question_attachment	varchar	DROPPED — read from qf_resource
question_answer_type	varchar	DROPPED — read from config: type
question_statement	text	DROPPED — read from qf_payload.body
marks_obtained	decimal	DROPPED — on qf_student_responses
attachment	varchar	DROPPED — on qf_student_responses

RESPONSES | MIGRATED

assessment_score

↳ *qf_student_responses.response_score + grader_id*

Scoring is now a direct column on the response row. No separate score table needed.

	Column	Type	Migration destination / note
PK	id	PK	DROPPED — score is on qf_student_responses
FK	student_submitted_assessment_id	FK	→ via qf_delivery_responses
	score	decimal	→ qf_student_responses.response_score
	scored_by	int	→ qf_student_responses.grader_id (urdd_id)
	paramter_id	int	→ future config key: rubric_param_id
	user_link_level_tag_id	int	→ resolved to student_id

RESPONSES | MIGRATED

feedback

↳ *qf_student_responses row where grader_id is set (comment as student_response)*

Teacher feedback is a response row from the grader's perspective. The comment becomes student_response; the attachment becomes a qf_resource linked via qf_content_delivery.

	Column	Type	Migration destination / note
PK	feedback_id	PK	→ qf_student_responses.student_response_id
	comment	text	→ qf_student_responses.student_response
	attachment	varchar	→ qf_resource + qf_content_delivery
	user_id	int	→ qf_student_responses.grader_id
FK	student_submitted_assessment_link_answer	FK	→ resolved to payload_id
	flag	tinyint	→ future config key: feedback_flag

RESPONSES MIGRATED

feedback_parameters

↳ *Future config keys on qf_student_responses (rubric scoring framework)*

Column	Type	Migration destination / note
PK id	PK	→ future: qf_config_keys entry
name	varchar	→ config_key name in qf_config_keys
description	text	→ qf_config_keys.description
status	varchar	→ config_key active/inactive flag

RESPONSES | MIGRATED

quiz_score

↳ *qf_student_responses (one row per MCQ answer) — totals derived at query time*

Quiz scores are no longer stored as a total. Each MCQ answer is one student_responses row. Total score = SUM(response_score). Total questions = COUNT(qf_delivery_payload WHERE delivery_id=x).

Column	Type	Migration destination / note
PK id	PK	DROPPED — per-answer rows used
user_link_level_tag_id	int	→ qf_student_responses.student_id
FK quiz_id	FK	→ qf_delivery_responses.delivery_id
total_questions	int	*DERIVED: COUNT(qf_delivery_payload WHERE delivery_id=x)*
score	decimal	*DERIVED: SUM(response_score) per student per delivery*
status	varchar	→ future config key: delivery_completion_status

RESPONSES MIGRATED

quiz_user_link_missing_level_tag

↳ *Future config key on qf_delivery_responses*

Column	Type	Migration destination / note
PK quiz_user_link_missing_level_tag_id	PK	—
user_id	int	→ student_id reference
level_tag_id	int	→ future config: missing_level_tag_id
semester_id	int	→ via qf_delivery.semester_id

RESPONSES | MIGRATED

user_link_lesson

↳ *qf_student_responses (payload_id=NULL) + qf_delivery_responses*

A lesson view is a student_responses row with payload_id=NULL, student_response=NULL, response_score=NULL. submitted_at records the view timestamp.

Column	Type	Migration destination / note
PK user_link_lesson_id	PK	→ qf_student_responses.student_response_id
user_link_level_tag_id	int	→ student_id (resolved)
FK lesson_id	FK	→ qf_delivery_responses.delivery_id

RESPONSES MIGRATED

notifications_settings_new

↳ *qf_config (target_table='qf_user_notification') — config rows keyed by notification type*

Column	Type	Migration destination / note
PK id	PK	→ qf_config_keys.config_key_id
title	varchar	→ config: notification_title
detail	text	→ config: notification_detail
type	varchar	→ config: notification_type
for_year	int	→ config: notification_for_year

RESPONSES MIGRATED

users_notifications_settings_new

↳ *qf_config key: notification_enabled (record_id = user_id)*

Column	Type	Migration destination / note
PK id	PK	DROPPED — config row used
user_id	int	→ qf_config.record_id
FK notification_setting_id	FK	→ config_key_id reference
flag	tinyint	→ config: notification_enabled (0/1)

RESPONSES MIGRATED

user_clear_notifications

↳ *qf_config key: notification_cleared_at (record_id = user_id)*

Column	Type	Migration destination / note
PK id	PK	DROPPED — config row used
user_id	int	→ qf_config.record_id
un_id	int	→ config: notification_instance_id

Dropped Tables

8 operational log tables → external log sink. No business data to migrate.

DROPPED DROPPED

debug_logs

↳ *DROPPED — move to external log sink (DataDog, CloudWatch, etc.)*

Column	Type	Migration destination / note
PK id	PK	—
message	text	→ log sink
type	varchar	—
created_at	datetime	—

DROPPED DROPPED

error_logs

↳ *DROPPED — move to external error tracking (Sentry, etc.)*

Column	Type	Migration destination / note
PK el_id	PK	—
el_data	text	→ error tracker
el_note	text	—
el_verb	varchar	—

DROPPED DROPPED

log

↳ *DROPPED — move to external log sink*

Column	Type	Migration destination / note
PK id	PK	—
level	varchar	→ log sink
category	varchar	—
message	text	—

DROPPED DROPPED

request_log

↳ *DROPPED — move to APM / request tracing tool*

Column	Type	Migration destination / note
PK id	PK	—
method	varchar	→ APM
url	varchar	—
response_status	int	—

DROPPED DROPPED

isoft_log

↳ *DROPPED — iSoft integration logs, move to external sink*

Column	Type	Migration destination / note
PK il_id	PK	—
il_data	text	→ log sink

DROPPED DROPPED

internal_launch_automation_logs

↳ *DROPPED — automation logs, move to external sink*

Column	Type	Migration destination / note
PK id	PK	—
message	text	→ log sink
type	varchar	—

DROPPED DROPPED

in_app_purhcase_logs

↳ *DROPPED — in-app purchase logs, move to payment monitoring tool*

Column	Type	Migration destination / note
PK id	PK	—
message	text	→ payment monitoring

DROPPED DROPPED

test

↳ *DROPPED — empty test table*

Column	Type	Migration destination / note
PK `id`	PK	—

New Schema

9 new tables that absorb all 60 eliminated tables

CORE EAV | NEW TABLE

qf_config_keys

Absorbs: *NEW — governs vocabulary for qf_config. Replaces ad-hoc key strings in application code.*

The controlled vocabulary registry. Every attribute key must be registered here before it can be stored in qf_config. Makes the database self-documenting.

Column	Type	Purpose / note
PK config_key_id	PK SMALLINT	Tiny lookup table — fits in memory
config_key	varchar(80)	Attribute slug, e.g. 'post_date', 'is_mcq'
target_table	varchar(80)	Which base_table this key applies to
value_type	varchar(20)	varchar | int | bool | decimal | datetime | text | json
description	text	Human-readable documentation of this attribute
created_at	datetime	—

CORE EAV | NEW TABLE

qf_config

Absorbs: *ALTERED — config_key_id FK added. Was already in old schema but uncontrolled.*

Universal EAV store. One row = one attribute on one record. All sparse/flexible metadata for every table lives here.

Column	Type	Purpose / note
PK id	PK	—
base_table	varchar	Name of the table this row's record belongs to
record_id	bigint	PK of the record in base_table
FK NEW config_key_id	FK NEW	→ qf_config_keys — enforces controlled vocabulary
config_key	varchar	Denormalised key name, kept for legacy read compat
config_value	text	Serialised value — cast to value_type on read

DELIVERY | NEW TABLE

qf_delivery_types

Absorbs: *NEW — extensible type registry. Replaces hard-coded content type logic.*

Pre-seeded with 11 types. Adding a new content type costs one INSERT — no schema change needed.

Column	Type	Purpose / note
PK delivery_type_id	PK TINYINT	Fits in a single byte
name	varchar(64)	lesson | quiz | assessment | ta_live_session | email_template | page_content | note | ta_recording | tutorial | welcome video | resource file
label	varchar(128)	Human-readable UI label
created_at	datetime	—

DELIVERY | NEW TABLE

qf_delivery

Absorbs: *lessons, assessment, quiz, ta_live_sessions, email_templates, page_content, notes, ta_recordings, tutorials, welcome_videos*

One row per deliverable unit of any type. Behaviour is entirely determined by delivery_type_id and qf_config rows.

	Column	Type	Purpose / note
PK	delivery_id	PK	—
FK	delivery_type_id	FK	→ qf_delivery_types — determines config key expectations
	title	varchar(512)	NOT NULL
	description	text	—
	owner_id	bigint	urdd_id of creating teacher/admin
	semester_id	bigint	FK → qf_semester (denormalised shortcut)
	created_at	datetime	—
	updated_at	datetime	—

DELIVERY | NEW TABLE

qf_payload

Absorbs: *question, mcq_question, mcq_question_option, email_templates.html, page_content.content, notes.body, mail_merger*

Atomic content unit. The 'type' config key governs rendering. One payload can be reused across deliveries (shared instructions).

Column	Type	Purpose / note
PK payload_id	PK	—
body	MEDIUMTEXT	Question stem | option text | HTML body | note body | NULL for mail variables
created_at	datetime	—
updated_at	datetime	—

CONTENT | NEW TABLE

qf_resource

Absorbs: *resource, support_links, ta_meeting_room_links+ta_schedule_timing (as ta_profile), lesson_link_video*

Universal content handle. resource_type config key determines the logical type. attachment_id is NULL for virtual resources (ta_profile, support_link).

Column	Type	Purpose / note
PK resource_id	PK	—
attachment_id	bigint	FK → file-storage table. NULL for virtual resources
created_at	datetime	—
updated_at	datetime	—

CONTENT | NEW TABLE

qf_delivery_payload

Absorbs: *assessment_link_question, quiz_mcq_question, all question-ordering logic*

Ordered bridge between a delivery and its payload items. sequence drives display order. A payload can be reused across deliveries.

	Column	Type	Purpose / note
PK	id	PK	—
FK	delivery_id	FK	→ qf_delivery
FK	payload_id	FK	→ qf_payload
	sequence	smallint	0-based display order within the delivery

CONTENT | NEW TABLE

qf_content_delivery

Absorbs: *lesson_semester_plans, resource_semester_plan, tutorial_semester_plan, assessment_semester_plans, all attachment FK columns*

Replaces ALL semester_plan bridge tables and inline attachment_id columns with one consistent pattern. One delivery ↔ many resources; one resource ↔ many deliveries.

	Column	Type	Purpose / note
PK	id	PK	—
FK	delivery_id	FK	→ qf_delivery
FK	resource_id	FK	→ qf_resource

RESPONSES | NEW TABLE

qf_student_responses

Absorbs: *answer, quiz_score, assessment_score, feedback (teacher comments), user_link_lesson (lesson views)*

One row per atomic answer or event. MCQ auto-graded immediately. Manual grading fills grader_id + graded_at. Lesson views use payload_id=NULL.

Column	Type	Purpose / note
PK student_response_id	PK	—
student_id	bigint	urdd_id — NOT NULL
FK payload_id	FK nullable	→ qf_payload. NULL for lesson-view events
student_response	MEDIUMTEXT	Answer text or attachment ref. NULL for lesson views
response_score	DECIMAL(10,4)	NULL until graded. Auto-set for MCQ; manual for assessments
grader_id	bigint nullable	urdd_id of grading teacher/TA. NULL until graded
submitted_at	datetime	NOT NULL DEFAULT NOW()
graded_at	datetime nullable	NULL until graded

RESPONSES | NEW TABLE

qf_delivery_responses

Absorbs: *student_submitted_assessment, student_submitted_assessment_link_answer (dropped entirely)*

Ties each student response to the delivery it came from. Enables grade-book, progress tracking, and analytics with a single join.

	Column	Type	Purpose / note
PK	id	PK	—
FK	delivery_id	FK	→ qf_delivery
FK	student_response_id	FK	→ qf_student_responses

System Flows

How content is authored, configured, delivered, and responded to — including hierarchy rules, temporal identity, and edge cases

System Flows — How It All Works

End-to-end walkthrough: content creation → configuration → delivery → student access → response → grading

1. Author content

Teacher creates qf_resource + qf_payload

→

2. Configure delivery

Set delivery type, audiencce, schedule → qf_config

→

3. Resolve audience

Config hierarchy: content cfg ⊂ delivery cfg rules

→

4. Student accesses

Visibility gate checked at runtime vs config keys

→

5. Response & grading

qf_student_responses MCQ auto / manual grading

Config hierarchy & override rules

Content config < Delivery config. Audience union vs override. Which wins and why.

Config change over time

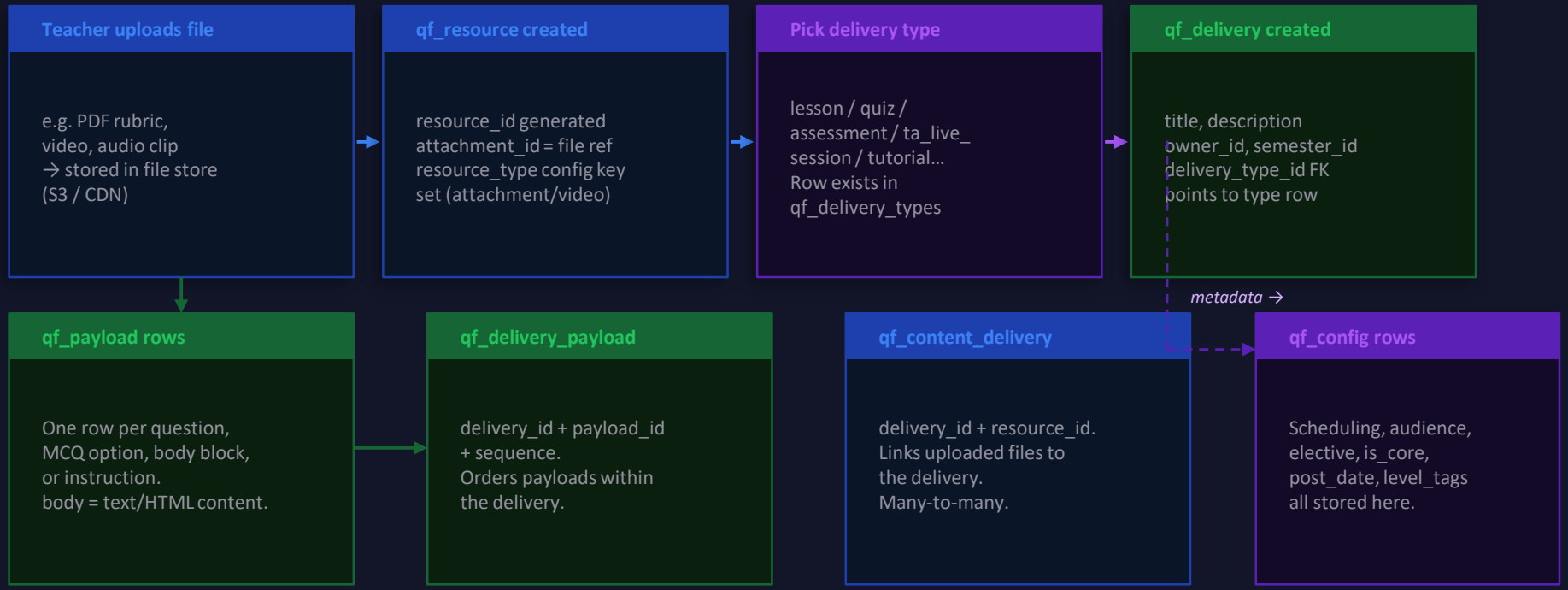
How config mutations are versioned, effective-from/until dating, and audit trail.

Temporal student identity

A user who was a child last year remains a child in all historical records, forever.

Flow 1 — Content Authoring

How a teacher authors a lesson, assessment, or quiz and attaches resources



Flow 2 — Audience Configuration

How demographic targeting (child, revert, year 1, gender, elective) is stored and resolved at runtime

RESOURCE-LEVEL CONFIG (content scope)

KEY	VALUE	EFFECT
visible_to_designation	'child'	Only students with designation=child
visible_to_semester_id	42	Only within semester 42
gender_scope	'female'	Female students only
available_from	'2024-09-01 00:00:00'	Not visible before this date
available_until	'2025-06-30 23:59:59'	Expires after this date
max_view_seconds	1800	30-min cap per session

DELIVERY-LEVEL CONFIG (delivery scope)

KEY	VALUE	EFFECT
designation_id	'revert'	Targets revert students only
elective_id	7	Elective track 7
elective_name	'Tajweed Advanced'	Denormalised label
is_core	0	Elective content, not core
post_date	'2024-10-15 09:00:00'	Appears in feed at this time
level_tags	'12,13'	Visible to level 2 and 3 students

How config is read at runtime

```
SELECT c.config_key, c.config_value, ck.value_type
FROM   qf_config c
JOIN   qf_config_keys ck ON ck.config_key_id = c.config_key_id
WHERE  c.base_table = 'qf_delivery' AND c.record_id = :delivery_id
   OR  c.base_table = 'qf_resource'  AND c.record_id = :resource_id;
```

Flow 3 — Config Hierarchy & Override Rules

Content config sets defaults. Delivery config can union or override. The runtime resolver applies a clear precedence chain.

qf_config_keys

Registered key catalogue
(value_type + description)

↓ overrides ↓

qf_resource config

Content defaults
(audience, availability, duration)

↓ overrides ↓

qf_delivery config

Delivery overrides
(designation, schedule, elective)

Config key	Resource value	Delivery value	Resolved value	Rule applied
visible_to_designation	child	revert	**revert**	*OVERRIDE — delivery wins*
gender_scope	female	(not set)	**female**	*INHERIT — resource value us*
available_from	2024-09-01	2024-10-01	**2024-10-01**	*OVERRIDE — delivery wins (l*
available_until	2025-06-30	(not set)	**2025-06-30**	*INHERIT — resource value us*
level_tags	11,12	12,13	**11,12,13**	*UNION — both sets merged*
elective_id	(not set)	7	**7**	*DELIVERY ONLY — no resourc*
max_view_seconds	1800	3600	**3600**	*OVERRIDE — delivery wins (l*

Override — delivery value replaces resource value

Union — both values are merged (e.g. tag arrays)

Inherit — resource value used w delivery has none

Flow 4 — Assigning Delivery to a Demographic

Example: assigning a lesson to child + revert students in semester 5, elective Tajweed, level 2 only

1 Create delivery row

```
INSERT INTO qf_delivery
  (delivery_type_id, title, owner_id, semester_id)
VALUES (1, 'Makhaarij Lesson 4', 12, 5);
```

2 Register audience config

```
INSERT INTO qf_config (base_table, record_id, config_key_id,
config_key, config_value)
VALUES
  ('qf_delivery', :id, 14, 'designation_id',  'child'),
  ('qf_delivery', :id, 14, 'designation_id',  'revert'),
  ('qf_delivery', :id, 17, 'elective_id',      '3'),
  ('qf_delivery', :id, 18, 'elective_name',    'Tajweed Basics'),
  ('qf_delivery', :id, 10, 'level_tags',       'l2'),
  ('qf_delivery', :id,  9, 'post_date',        '2024-10-07 09:00:00'),
  ('qf_delivery', :id, 11, 'is_core',          '0');
```

3 Attach resource (PDF)

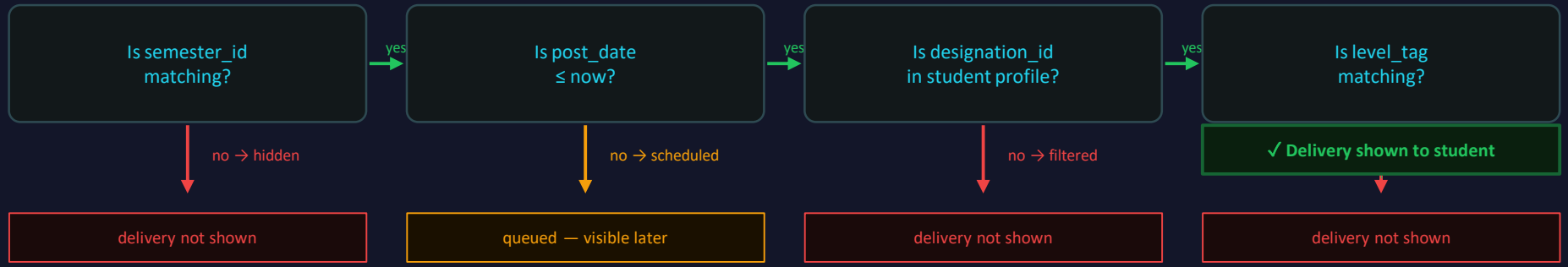
```
INSERT INTO qf_content_delivery (delivery_id, resource_id)
VALUES (:delivery_id, :resource_id);
```

4 Add payload (question)

```
INSERT INTO qf_payload (body) VALUES ('Describe the seven
makhaarij...');
INSERT INTO qf_delivery_payload (delivery_id, payload_id, sequence)
VALUES (:delivery_id, :payload_id, 0);
```

Flow 5 — Runtime Visibility Gate

How the system decides whether a student can see a delivery — every access request runs this check



Additional runtime checks

`available_from / available_until`	compared to NOW() server time — datetime range gate
`gender_scope`	compared to student profile gender field (M/F/any)
`is_core / elective_id`	matched against student's is_core_enrollment + elective_id in user_link_level_tag
`max_view_seconds`	enforced client-side + server session timer — not a visibility gate

Flow 6 — Student Response Handling

How responses are captured, auto-graded, manually graded, and stored — for lessons, quizzes, and assessments

Lesson view

Student opens lesson

Visibility gate passes

```
INSERT qf_student_responses:
  student_id = :urdd_id
  payload_id = NULL
  student_response = NULL
  response_score = NULL
```

```
INSERT qf_delivery_responses:
  delivery_id = :lesson_id
  student_response_id = :new_id
```

Progress tracked. No grading needed.

Quiz (MCQ)

Student submits MCQ answer

App reads correct_option_id from qf_config on payload

```
INSERT qf_student_responses:
  payload_id = :question_payload_id
  student_response = 'B'
  submitted_at = NOW()
```

Auto-grade: compare response to correct_option_id →
response_score = 1.0 (correct)
response_score = 0.0 (wrong)

```
UPDATE qf_student_responses
  SET response_score = :score
  WHERE student_response_id = :id
```

```
INSERT qf_delivery_responses
  (delivery_id, student_response_id)
```

Assessment (open answer)

Student submits written answer (or attachment reference)

```
INSERT qf_student_responses:
  payload_id = :question_payload_id
  student_response = 'text…'
  response_score = NULL
  grader_id = NULL
```

```
INSERT qf_delivery_responses
  (delivery_id, student_response_id)
```

```
Teacher/TA reviews in gradebook:
  SELECT * FROM qf_delivery_responses
  WHERE delivery_id = :id
```

```
Teacher grades:
  UPDATE qf_student_responses
  SET response_score = 8.5,
      grader_id = :teacher_urdd_id,
      graded_at = NOW()
```

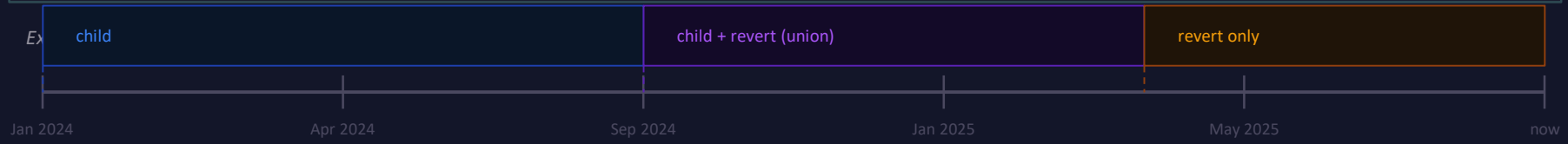
```
Optional: INSERT feedback row
  (grader comment as separate
   student_response where
   grader_id is set)
```

Flow 7 — Config Changes Over Time

How the system handles config mutations: effective dating, version history, and audit trail

qf_config extended for temporal support

```
ALTER TABLE qf_config ADD COLUMN  effective_from  DATETIME  DEFAULT NULL   -- when this config value became active
ALTER TABLE qf_config ADD COLUMN  effective_until DATETIME  DEFAULT NULL   -- NULL = still active
ALTER TABLE qf_config ADD COLUMN  changed_by      BIGINT    DEFAULT NULL   -- urdd_id of admin who made the change
```



Query active config at any point in time

```
SELECT * FROM qf_config
WHERE base_table = 'qf_delivery'
  AND record_id = :delivery_id
  AND (effective_from  IS NULL OR effective_from  <=
:point_in_time)
  AND (effective_until IS NULL OR effective_until >
:point_in_time);
```

Mutation pattern — never UPDATE, always INSERT

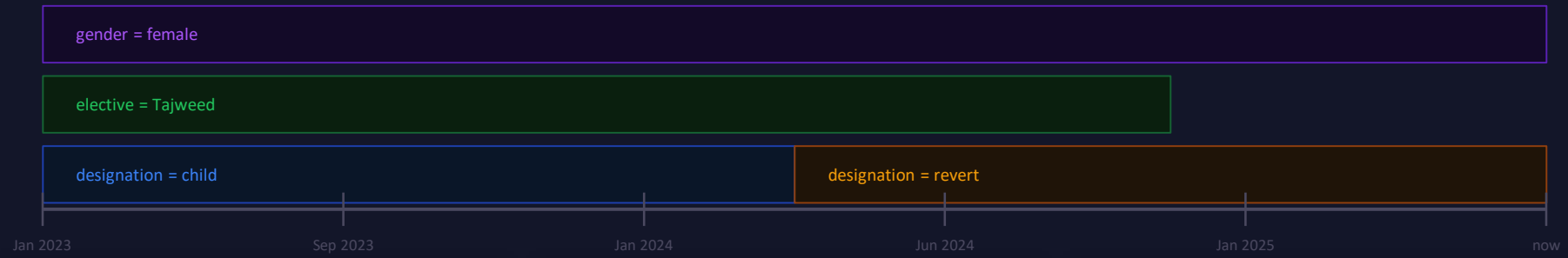
```
-- 1. Close existing value
UPDATE qf_config SET effective_until = NOW()
WHERE record_id = :id AND config_key = 'designation_id'
  AND effective_until IS NULL;
-- 2. Insert new value
INSERT INTO qf_config (..., config_value, effective_from,
changed_by)
VALUES (..., 'revert', NOW(), :admin_id);
```

Key insight: config values are never deleted or overwritten — they are closed with an effective_until timestamp and replaced by a new row. Full history is always preserved. A point-in-time query reconstructs the exact state of any delivery at any moment in the past.

Flow 8 — Temporal Student Identity

A student who was a child 12 months ago does not stop being a child in the history of the application

Student #1047 — identity timeline



Scenario A — child views lesson (Jan 2023)

Lesson had config: designation_id = 'child'

Student had designation = 'child' at that time

Visibility gate: PASS

qf_student_responses row created → submitted_at = Jan 2023

record is forever tied to the 'child' period

Even after student becomes 'revert', this response still shows as a child-era submission

Scenario B — point-in-time grade report

Admin runs grade report for semester 2023-S1

Query: WHERE submitted_at BETWEEN '2023-01-01' AND '2023-08-31'

Joins to user_link_level_tag WHERE semester_id = 2023-S1

Result: student appears as 'child' with their child-era scores

No data is retroactively changed when designation changes

Historical integrity is preserved by immutable response rows

Scenario C — what changed and when

Query qf_config for the student's delivery-response config:

WHERE config_key = 'designation_id'

AND effective_until IS NOT NULL (closed = historical)

Each row shows what the designation WAS and when it closed

Current designation: WHERE effective_until IS NULL

Full audit: all rows ordered by effective_from

Golden rule: Response rows are immutable. They record what was true at submission time. A student's current designation does not rewrite the past. Historical identity is preserved in `qf_student_responses.submitted_at` + `user_link_level_tag.semester_id`.

Flow 9 — Config Edge Cases

Unusual but real-world scenarios: conflicting configs, retroactive changes, missing keys, expired content

Content rated for children, delivery aimed at adults

DELIVERY WINS

qf_resource config: designation_id = 'child'
qf_delivery config: designation_id = 'adult'

Runtime resolver: delivery config takes precedence. The delivery's audience overrides the content's default audiencce. The resource is shown to adult students.

Rationale: a delivery always has more specific context than the resource it reuses.

Delivery has no audience config set

INHERIT FROM RESOURCE

qf_delivery config: designation_id = (not set)
qf_resource config: designation_id = 'child'

Runtime resolver: fall back to resource config. Delivery inherits the content's audience restriction.

Rationale: a delivery without an explicit audience defaults to the content's declared audience.

Both resource and delivery have level_tags

UNION (SET MERGE)

qf_resource config: level_tags = 'l1,l2'
qf_delivery config: level_tags = 'l2,l3'

Runtime resolver: UNION → 'l1,l2,l3'
Tags are additive — a student matching ANY tag in the merged set can see the delivery.

Rationale: tags widen audience, they don't restrict it. Deliver to the broadest valid set.

Config key missing from qf_config_keys

REJECT AT INSERT

Attempting to INSERT a row in qf_config with a config_key_id that doesn't exist in qf_config_keys will fail the FK constraint — hard error at the DB layer.

The application must first INSERT the new key into qf_config_keys, then use the returned ID. This enforces the controlled vocabulary at all times.

Content expires mid-semester

RUNTIME GATE

qf_resource config: available_until = '2025-03-01'
Student submits assessment on '2025-03-15'

Runtime gate: available_until < NOW() → FAIL
Delivery is not shown. If student already submitted a response before expiry, that response is preserved — only new access is blocked. Past responses are never retroactively invalidated.

Config retroactively changed for a past semester

TEMPORAL QUERY PRESERVES HISTORY

Admin changes designation_id from 'child' to 'revert' for a delivery that ran in semester 2023-S1.

The mutation pattern: close old row (effective_until=NOW())
and insert new row (effective_from=NOW()).

Past responses (submitted_at < NOW()) are unaffected. Historical queries using effective_from/until correctly return 'child' for the 2023-S1 period.

Flow 10 — Full End-to-End Query Example

Retrieve all deliveries visible to a specific student, with their responses, at a given point in time

Step 1 — Resolve student profile

```
SELECT u.user_id, u.user_type, ullt.level_tag_id,
       ullt.semester_id, ullt.elective_id, ullt.is_core_enrollment,
       up.gender
FROM   users u
JOIN   user_link_level_tag ullt ON ullt.user_id  = u.user_id
JOIN   user_profile        up   ON up.user_id    = u.user_id
WHERE  u.user_id = :student_id
  AND  ullt.semester_id = :current_semester;
```

Step 2 — Fetch visible deliveries (config-gated)

```
SELECT d.delivery_id, d.title, dt.name AS delivery_type,
       MAX(CASE WHEN ck.config_key='post_date'        THEN c.config_value END) AS post_date,
       MAX(CASE WHEN ck.config_key='designation_id'   THEN c.config_value END) AS designation,
       MAX(CASE WHEN ck.config_key='level_tags'       THEN c.config_value END) AS level_tags
FROM   qf_delivery d
JOIN   qf_delivery_types dt ON dt.delivery_type_id = d.delivery_type_id
JOIN   qf_config c  ON c.base_table = 'qf_delivery' AND c.record_id = d.delivery_id
JOIN   qf_config_keys ck ON ck.config_key_id = c.config_key_id
WHERE  d.semester_id = :semester_id
  AND  (c.effective_from  IS NULL OR c.effective_from  <= :now)
  AND  (c.effective_until IS NULL OR c.effective_until >  :now)
GROUP  BY d.delivery_id
HAVING designation IS NULL OR FIND_IN_SET(:student_designation, designation) > 0
```

Migration Summary

82 Tables before	**22** Core tables after	**60** Tables eliminated
9 New tables added	**50+** Config keys registered	**0** Schema changes for new types

Key design principles

1. **qf_config_keys enforces a controlled vocabulary** — every attribute key must be registered before use, making the DB self-documenting.
2. **qf_config is the universal EAV store** — all flexible/sparse metadata lives here. One row = one attribute on one record.
3. **New content types cost one INSERT** — add a row to qf_delivery_types and qf_config_keys. Zero schema migrations.
4. **Payloads are immutable after publish** — no snapshot columns needed. student_submitted_assessment_link_answer is dropped entirely.